09. 실제 사용 예제 및 벤치마크

목차

1. 기본 사용 예제
2. 성능 비교 벤치마크
3. 네트워크 조건별 테스트
4. 비잔틴 시뮬레이션
5. 프로덕션 배포 예제
6. 모니터링 및 알림 설정

기본 사용 예제

간단한 Autobahn 노드 설정

```rust
// examples/basic_autobahn.rs

use std::sync::Arc;
use std::time::Duration;
use config::{Committee, Parameters, ConsensusAlgorithm, AutobahnParameters};
use crypto::{KeyPair, NetworkKeyPair};
use narwhal_node::{Node, NodeStorage};
use prometheus::Registry;
```

```rust
#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
// 로깅 초기화
tracing_subscriber::fmt()
.with_max_level(tracing::Level::INFO)
.init();

println!("🚀 Starting Autobahn BFT Example");

// Autobahn 설정 생성
let autobahn_params = AutobahnParameters {
fast_path_timeout: Duration::from_millis(100),
max_fast_path_timeout: Duration::from_secs(2),
min_fast_path_timeout: Duration::from_millis(50),
timeout_adaptation_factor: 0.15,
timeout_history_size: 50,
message_batch_size: 50,
message_batch_timeout: Duration::from_millis(25),
enable_byzantine_detection: true,
byzantine_threshold: 1,
};
```

```rust
// Autobahn용 매개변수 설정
let mut parameters = Parameters::default();
parameters.consensus_algorithm = ConsensusAlgorithm::Autobahn;
parameters.autobahn = Some(autobahn_params);

// 빠른 예제 실행을 위한 타임아웃 감소
parameters.max_header_delay = Duration::from_millis(50);
parameters.max_batch_delay = Duration::from_millis(50);

// 4노드 위원회 생성 (3f+1에서 f=1)
let committee = create_test_committee(4).await?;
let worker_cache = create_test_worker_cache(&committee).await?;

// 실행 상태 생성
let execution_state = Arc::new(ExampleExecutionState::new());

// 저장소 생성
let store_path = tempfile::tempdir()?;
let storage = NodeStorage::reopen(store_path.path());

// Prometheus 레지스트리 생성
let registry = Registry::new();
```

```
// 첫 번째 노드용 키페어 생성
let  keypair = KeyPair::generate(&mut rand::thread_rng());
let  network_keypair = NetworkKeyPair::generate(&mut rand::thread_rng());


println!("🌟 Starting Autobahn consensus node...");
// Autobahn 컨센서스로 Primary 노드 스폰
let  handles = Node::spawn_primary(
keypair,
network_keypair,
committee.clone(),
worker_cache.clone(),
&storage,
parameters.clone(),
true, // 내부 컨센서스 사용
execution_state.clone(),
&registry,
).await?;


println!("✅ Autobahn node started successfully!");
println!("🔄 Node is now running with Autobahn BFT consensus");


// 컨센서스 시연을 위해 30초 동안 실행
println!("⏳ Running for 30 seconds to demonstrate consensus...");
```

```
for i in 1..=30 {
tokio::time::sleep(Duration::from_secs(1)).await;
let processed = execution_state.get_processed_count();
if i % 5 == 0 {
println!("📊 Status after {}s: {} certificates processed", i, processed);
}
}

println!("🏁 Example completed!");
println!("📈 Final statistics:");
println!(" - Total certificates processed: {}", execution_state.get_processed_co

// 정상 종료
println!("🛑 Shutting down...");
for handle in handles {
handle.abort();
}

Ok(())
}
```#
성능 비교 벤치마크

벤치마크 테스트 구성

```rust
// examples/performance_comparison.rs


use criterion::{criterion_group, criterion_main, Criterion, BenchmarkId};
use std::time::Duration;
use tokio::runtime::Runtime;


/// Fast Path 컨센서스 성능 벤치마크
fn bench_fast_path_consensus(c: &mut Criterion) {
let rt = Runtime::new().unwrap();
let mut group = c.benchmark_group("fast_path_consensus");
// 다양한 배치 크기 테스트
for batch_size in [10, 50, 100, 200].iter() {
group.bench_with_input(
BenchmarkId::new("batch_size", batch_size),
batch_size,
|b, &batch_size| {
b.to_async(&rt).iter(|| async {
let config = create_test_config_with_batch_size(batch_size);
let controller = create_test_controller(config).await;
// 인증서 처리 시뮬레이션
for _ in 0..10 {
```

```
let  certificate = create_dummy_certificate();
let  _ = process_certificate(&controller, certificate).await;
}
});
},
);
}
group.finish();
}


/// 성능 비교 테스트 설정
#[derive(Debug, Clone)]
struct TestConfig {
name: String,
consensus_algorithm: ConsensusAlgorithm,
autobahn_params: Option<AutobahnParameters>,
test_duration: Duration,
expected_improvement: f64,
}


impl TestConfig {
fn  narwhal() -> Self {
Self {
```

```
name: "Narwhal (Baseline)".to_string(),
consensus_algorithm: ConsensusAlgorithm::Narwhal,
autobahn_params: None,
test_duration: Duration::from_secs(60),
expected_improvement: 1.0, // 기준선
}
}

fn  autobahn_conservative() -> Self {
Self {
name: "Autobahn (Conservative)".to_string(),
consensus_algorithm: ConsensusAlgorithm::Autobahn,
autobahn_params: Some(AutobahnParameters {
fast_path_timeout: Duration::from_millis(200),
timeout_adaptation_factor: 0.05, // 보수적 적응
message_batch_size: 50,
// ... 기타 설정
}),
test_duration: Duration::from_secs(60),
expected_improvement: 1.15, // 15% 향상 예상
}
}
```

```
fn autobahn_aggressive() -> Self {
Self {
name: "Autobahn (Aggressive)".to_string(),
consensus_algorithm: ConsensusAlgorithm::Autobahn,
autobahn_params: Some(AutobahnParameters {
fast_path_timeout: Duration::from_millis(50),
timeout_adaptation_factor: 0.2, // 공격적 적응
message_batch_size: 100,
// ... 기타 설정
}),
test_duration: Duration::from_secs(60),
expected_improvement: 1.25, // 25% 향상 예상
}
}
}

async fn run_performance_test(config: &TestConfig) -> Result<PerformanceMetric
println!("🚀 Starting test: {}", config.name);

// 매개변수 생성
let mut parameters = Parameters::default();
parameters.consensus_algorithm = config.consensus_algorithm.clone();
parameters.autobahn = config.autobahn_params.clone();
```

```rust
// 성능 테스트를 위한 최적화
parameters.max_header_delay = Duration::from_millis(10);
parameters.max_batch_delay = Duration::from_millis(10);

// 테스트 환경 생성
let committee = create_test_committee(4).await?;
let worker_cache = create_test_worker_cache(&committee).await?;
let execution_state = Arc::new(PerformanceTrackingState::new());

// 임시 저장소 생성
let store_path = tempfile::tempdir()?;
let storage = NodeStorage::reopen(store_path.path());

// 레지스트리 생성
let registry = Registry::new();

// 키페어 생성
let keypair = KeyPair::generate(&mut rand::thread_rng());
let network_keypair = NetworkKeyPair::generate(&mut rand::thread_rng());

println!("🌟 Starting consensus node...");
```

```
// 노드 스폰
let handles = Node::spawn_primary(
keypair,
network_keypair,
committee,
worker_cache,
&storage,
parameters,
true,
execution_state.clone(),
&registry,
).await?;

println!("⏱️ Running test for {:?}...", config.test_duration);

// 테스트 실행
tokio::time::sleep(config.test_duration).await;

// 최종 메트릭 가져오기
let final_metrics = execution_state.get_metrics();

// 종료
println!("🛑 Shutting down test...");
```

```
for handle in handles {
handle.abort();
}

Ok(final_metrics)
}

fn print_comparison_results(results: &[(TestConfig, PerformanceMetrics)]) {
println!("\n" + "=".repeat(80));
println!("📈 PERFORMANCE COMPARISON RESULTS");
println!("=".repeat(80));

if results.is_empty() {
println!("No results to compare.");
return;
}

// 첫 번째 결과를 기준선으로 사용 (Narwhal이어야 함)
let baseline = &results[0];
let baseline_throughput = baseline.1.throughput();
let baseline_latency = baseline.1.average_latency();

println!("\n📊 Throughput Comparison (certificates/sec):");
```

```
println!("{:<25} {:>15} {:>15} {:>15}", "Configuration", "Throughput", "vs Basel
println!("-".repeat(70));

for (config, metrics) in  results {
let  throughput = metrics.throughput();
let  improvement = if  baseline_throughput > 0.0 {
throughput / baseline_throughput
} else {
0.0
};

println!(
"{:<25} {:>15.2} {:>14.2}x {:>14.2}x",
config.name,
throughput,
improvement,
config.expected_improvement
);
}

println!("\n⏱ Latency Comparison (milliseconds):");
println!("{:<25} {:>15} {:>15} {:>15}", "Configuration", "Avg Latency", "vs Base
println!("-".repeat(70));
```

```
for (config, metrics) in  results {
let  avg_latency = metrics.average_latency();
let  latency_ratio = if  baseline_latency > Duration::ZERO {
avg_latency.as_secs_f64() / baseline_latency.as_secs_f64()
} else {
0.0
};


println!(
"{:<25} {:>12.2}ms {:>14.2}x {:>6.1}/{:.1}ms",
config.name,
avg_latency.as_secs_f64() * 1000.0,
latency_ratio,
metrics.min_latency().as_secs_f64() * 1000.0,
metrics.max_latency().as_secs_f64() * 1000.0,
);
}
}
```

네트워크 조건별 테스트

네트워크 시뮬레이션

```rust
// tests/network_conditions.rs

use std::time::Duration;
use tokio::time::sleep;

pub struct NetworkSimulator {
latency: Duration,
packet_loss: f64,
jitter: Duration,
}

impl NetworkSimulator {
pub fn excellent() -> Self {
Self {
latency: Duration::from_millis(5),
packet_loss: 0.001, // 0.1%
jitter: Duration::from_millis(1),
}
}

pub fn good() -> Self {
```

```
Self {
latency: Duration::from_millis(25),
packet_loss: 0.01, // 1%
jitter: Duration::from_millis(5),
}
}

pub fn poor() -> Self {
Self {
latency: Duration::from_millis(200),
packet_loss: 0.05, // 5%
jitter: Duration::from_millis(50),
}
}

pub async fn simulate_network_delay(&self) {
// 기본 지연시간
sleep(self.latency).await;
// 지터 추가
if self.jitter > Duration::ZERO {
let jitter_amount = rand::random::<f64>() * self.jitter.as_millis() as f64;
sleep(Duration::from_millis(jitter_amount as u64)).await;
}
```

```
// 패킷 손실 시뮬레이션
if rand::random::<f64>() < self.packet_loss {
// 패킷 손실로 인한 추가 지연
sleep(self.latency * 3).await;
}
}
}

#[tokio::test]
async fn test_autobahn_under_network_stress() {
let network_conditions = [
("Excellent", NetworkSimulator::excellent()),
("Good", NetworkSimulator::good()),
("Poor", NetworkSimulator::poor()),
];

for (condition_name, simulator) in network_conditions {
println!("Testing under {} network conditions", condition_name);
let mut controller = create_test_autobahn_controller().await;
let mut metrics = Vec::new();
for i in 0..100 {
let start = std::time::Instant::now();
// 네트워크 지연 시뮬레이션
```

```
simulator.simulate_network_delay().await;

// 인증서 처리

let  certificate = create_test_certificate(i);

let  result = controller.process_certificate(certificate).await;

let  duration = start.elapsed();

metrics.push((duration, result.is_ok()));

// 네트워크 지연시간 기록

controller.record_network_latency(duration).await.unwrap();

}

// 결과 분석

let  avg_latency: Duration = metrics.iter().map(|(d, _)| *d).sum::<Duration>() /

let  success_rate = metrics.iter().filter(|(_, success)| *success).count() as f6

println!(" Average latency: {:?}", avg_latency);

println!(" Success rate: {:.2}%", success_rate * 100.0);

// 성능 기준 확인

match  condition_name {

"Excellent" => {

assert!(avg_latency < Duration::from_millis(50));

assert!(success_rate > 0.95);

}

"Good" => {

assert!(avg_latency < Duration::from_millis(100));

assert!(success_rate > 0.90);
```

```
}
"Poor" => {
assert!(avg_latency < Duration::from_millis(500));
assert!(success_rate > 0.80);
}
_ => {}
}
}
}
```

비잔틴 시뮬레이션

비잔틴 노드 시뮬레이션

```
// tests/byzantine_simulation.rs

pub struct ByzantineSimulator {
byzantine_nodes: Vec<NodeId>,
attack_type: ByzantineAttackType,
}

pub enum ByzantineAttackType {
Equivocation, // 이중 투표
```

```
Withholding, // 메시지 보류
RandomDelay, // 무작위 지연
CoordinatedAttack, // 조정된 공격
}

impl ByzantineSimulator {
pub fn new(byzantine_nodes: Vec<NodeId>, attack_type: ByzantineAttackType) ->
Self {
byzantine_nodes,
attack_type,
}
}

pub async fn simulate_byzantine_behavior(&self, round: Round) -> Vec<Byzantin
let mut events = Vec::new();

for node_id in &self.byzantine_nodes {
match self.attack_type {
ByzantineAttackType::Equivocation => {
// 같은 라운드에 대해 두 개의 다른 인증서 생성
let cert1 = create_certificate(node_id, round, "data1");
let cert2 = create_certificate(node_id, round, "data2");
events.push(ByzantineEvent::Equivocation {
```

```
node: *node_id,
round,
certificates: vec![cert1.digest(), cert2.digest()],
});
}
ByzantineAttackType::Withholding => {
// 인증서 전송 지연 또는 생략
events.push(ByzantineEvent::Withholding {
node: *node_id,
round,
delay: Duration::from_secs(10),
});
}
ByzantineAttackType::RandomDelay => {
// 무작위 지연 추가
let  delay = Duration::from_millis(rand::random::<u64>() % 1000);
tokio::time::sleep(delay).await;
}
ByzantineAttackType::CoordinatedAttack => {
// 여러 노드가 협력하여 공격
if  self.byzantine_nodes.len() > 1 {
events.push(ByzantineEvent::CoordinatedAttack {
participants: self.byzantine_nodes.clone(),
```

```
attack_pattern: "delayed_consensus".to_string(),
});
}
}
}
}

events
}
}

#[tokio::test]
async fn test_byzantine_fault_tolerance() {
// 4노드 시스템에서 1개 비잔틴 노드 (f=1)
let total_nodes = 4;
let byzantine_count = 1;
let committee = create_test_committee(total_nodes).await.unwrap();
let byzantine_nodes = vec![NodeId(0)]; // 첫 번째 노드를 비잔틴으로 설정
let simulator = ByzantineSimulator::new(
byzantine_nodes,
ByzantineAttackType::Equivocation,
);
let mut controller = create_test_autobahn_controller_with_committee(committee)
```

```
// 여러 라운드에 걸쳐 비잔틴 행동 시뮬레이션
for round in 1..=10 {
let byzantine_events = simulator.simulate_byzantine_behavior(Round(round)).awai
// 비잔틴 이벤트 처리
for event in byzantine_events {
match event {
ByzantineEvent::Equivocation { node, round, certificates } => {
// 이중 투표 감지 테스트
for cert_digest in certificates {
let detected = controller.detect_equivocation(&node, round, &cert_digest).await
assert!(detected, "Equivocation should be detected");
}
}
_ => {}
}
}
// 시스템이 여전히 진행되는지 확인
let consensus_output = controller.generate_consensus_output(Round(round)).await
assert!(consensus_output.is_some(), "Consensus should continue despite Byzantine
}
// 최종 상태 확인
let metrics = controller.get_metrics().await;
assert!(metrics.byzantine_events_detected() > 0, "Byzantine events should be det
```

```
assert!(metrics.fast_path_usage_ratio() < 1.0, "Should fallback to slow path due

}
```

프로덕션 배포 예제

Docker Compose 설정

```yaml
# docker-compose.yml

version: '3.8'


services:

autobahn-node-1:

build: .

environment:

- NODE_ID=1

- CONSENSUS_ALGORITHM=Autobahn

- FAST_PATH_TIMEOUT=150ms

- BYZANTINE_THRESHOLD=1

volumes:

- ./config/node1.json:/app/config.json

- node1-data:/app/data

ports:

- "8001:8000"
```

```yaml
    - "9001:9000"
    networks:
    - autobahn-network


  autobahn-node-2:
    build: .
    environment:
    - NODE_ID=2
    - CONSENSUS_ALGORITHM=Autobahn
    volumes:
    - ./config/node2.json:/app/config.json
    - node2-data:/app/data
    ports:
    - "8002:8000"
    - "9002:9000"
    networks:
    - autobahn-network


  autobahn-node-3:
    build: .
    environment:
    - NODE_ID=3
    - CONSENSUS_ALGORITHM=Autobahn
```

```yaml
volumes:
- ./config/node3.json:/app/config.json
- node3-data:/app/data
ports:
- "8003:8000"
- "9003:9000"
networks:
- autobahn-network


autobahn-node-4:
build: .
environment:
- NODE_ID=4
- CONSENSUS_ALGORITHM=Autobahn
volumes:
- ./config/node4.json:/app/config.json
- node4-data:/app/data
ports:
- "8004:8000"
- "9004:9000"
networks:
- autobahn-network
```

```yaml
prometheus:
image: prom/prometheus:latest
ports:
- "9090:9090"
volumes:
- ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
networks:
- autobahn-network


grafana:
image: grafana/grafana:latest
ports:
- "3000:3000"
environment:
- GF_SECURITY_ADMIN_PASSWORD=admin
volumes:
- ./monitoring/grafana-dashboards:/var/lib/grafana/dashboards
networks:
- autobahn-network


volumes:
node1-data:
node2-data:
```

```yaml
node3-data:

node4-data:



networks:

autobahn-network:

driver: bridge
```

Kubernetes 배포

```yaml
# k8s/autobahn-deployment.yaml

apiVersion: apps/v1

kind: StatefulSet

metadata:

name: autobahn-consensus

spec:

serviceName: autobahn-consensus

replicas: 4

selector:

matchLabels:

app: autobahn-consensus

template:

metadata:

labels:
```

```yaml
app: autobahn-consensus

spec:

containers:

- name: autobahn-node

image: autobahn-bft:latest

ports:

- containerPort: 8000

name: consensus

- containerPort: 9000

name: worker

- containerPort: 9090

name: metrics

env:

- name: CONSENSUS_ALGORITHM

value: "Autobahn"

- name: POD_NAME

valueFrom:

fieldRef:

fieldPath: metadata.name

volumeMounts:

- name: config

mountPath: /app/config

- name: data
```

```yaml
mountPath: /app/data

resources:

requests:

memory: "1Gi"

cpu: "500m"

limits:

memory: "2Gi"

cpu: "1000m"

volumes:

- name: config

configMap:

name: autobahn-config

volumeClaimTemplates:

- metadata:

name: data

spec:

accessModes: ["ReadWriteOnce"]

resources:

requests:

storage: 10Gi
```

모니터링 및 알림 설정

Prometheus 설정

```yaml
# monitoring/prometheus.yml
global:
scrape_interval: 15s

scrape_configs:
- job_name: 'autobahn-consensus'
static_configs:
- targets:
- 'autobahn-node-1:9090'
- 'autobahn-node-2:9090'
- 'autobahn-node-3:9090'
- 'autobahn-node-4:9090'
scrape_interval: 5s
metrics_path: /metrics

rule_files:
- "autobahn_alerts.yml"

alerting:
alertmanagers:
- static_configs:
```

```
- targets:
- alertmanager:9093
```

Grafana 대시보드

```
{
"dashboard": {
"title": "Autobahn BFT Consensus Monitoring",
"panels": [
{
"title": "Fast Path Usage Ratio",
"type": "stat",
"targets": [
{
"expr": "autobahn_fast_path_usage_ratio",
"legendFormat": "{{instance}}"
}
]
},
{
"title": "Consensus Latency",
"type": "graph",
"targets": [
```

```
{
"expr": "histogram_quantile(0.95, autobahn_consensus_latency_seconds_bucket)",
"legendFormat": "95th percentile"
},
{
"expr": "histogram_quantile(0.50, autobahn_consensus_latency_seconds_bucket)",
"legendFormat": "50th percentile"
}
]
},
{
"title": "Byzantine Events",
"type": "graph",
"targets": [
{
"expr": "rate(autobahn_byzantine_events_total[5m])",
"legendFormat": "{{instance}}"
}
]
},
{
"title": "Mode Transitions",
"type": "graph",
```

```
"targets": [
{
"expr": "rate(autobahn_mode_transitions_total[5m])",
"legendFormat": "{{from}}_to_{{to}}"
}
]
}
]
}
}
```

알림 규칙

```yaml
# monitoring/autobahn_alerts.yml
groups:
- name: autobahn_consensus
rules:
- alert: HighConsensusLatency
expr: histogram_quantile(0.95, autobahn_consensus_latency_seconds_bucket) > 0.5
for: 2m
labels:
severity: warning
annotations:
```

```yaml
summary: "High consensus latency detected"
description: "95th percentile consensus latency is {{ $value }}s"

- alert: LowFastPathUsage
expr: autobahn_fast_path_usage_ratio < 0.5
for: 5m
labels:
severity: warning
annotations:
summary: "Low fast path usage"
description: "Fast path usage ratio is {{ $value }}"

- alert: ByzantineEventsDetected
expr: increase(autobahn_byzantine_events_total[5m]) > 0
for: 0m
labels:
severity: critical
annotations:
summary: "Byzantine events detected"
description: "{{ $value }} Byzantine events detected in the last 5 minutes"

- alert: FrequentModeTransitions
expr: rate(autobahn_mode_transitions_total[5m]) > 0.1
```

```yaml
for: 2m

labels:

severity: warning

annotations:

summary: "Frequent mode transitions"

description: "Mode transition rate is {{ $value }} per second"
```

이러한 예제와 벤치마크를 통해 Autobahn BFT의 실제 성능과 다양한 네트워크 조건에서의 동작을 확인할 수 있으며, 프로덕션 환경에서의 배포와 모니터링을 위한 완전한 가이드를 제공합니다.